State Sync Temelleri

Bu bölümde, multiplayer oyunlarda oyun dünyasının nasıl senkronize edildiğini (state synchronization) inceleyeceğiz. İki temel yaklaşım olan "Full State Sync" ve "Delta State Sync" arasındaki farkları, tick rate'in bu sürece etkisini ve bir sunucunun tüm oyuncu pozisyonlarını nasıl broadcast ettiğini detaylı olarak ele alacağız.

---

1. State Sync Nedir? (Durum Senkronizasyonu)

State sync (durum senkronizasyonu), sunucudaki oyun dünyasının güncel halinin (state) istemcilere iletilmesi sürecidir. Multiplayer oyunlarda, her oyuncunun ekranında tutarlı bir oyun dünyası görmesi için bu işlem hayati öneme sahiptir.

1.1. State (Durum) Nedir?

State, oyun dünyasının belirli bir andaki tüm özelliklerini içeren veri bütünüdür:

- Her oyuncunun konumu (x, y, z)
- Her oyuncunun rotasyonu, hızı, canı (health)
- Nesnelerin durumları (kırılan duvarlar, açılan kapılar)
- Mermiler, patlamalar, efektler
- Puan durumu, süre, oyun modu bilgileri

1.2. State Sync'in Temel Prensipleri

1. **Yetki (Authority):** Oyun dünyasının tek bir "doğru" kopyası sunucuda bulunur.
2. **Dağıtım (Distribution):** Sunucu, bu doğru durumu düzenli aralıklarla tüm istemcilere gönderir.
3. **Görselleştirme (Rendering):** İstemciler, aldıkları bu durumu kendi ekranlarında render eder.
4. **Tutarlılık (Consistency):** Tüm istemciler aynı oyun dünyasını (küçük gecikme farklarıyla) görür.

---

2. Full State Sync (Tam Durum Senkronizasyonu)

Full state sync, sunucunun oyun dünyasının **tamamını** (tüm oyuncular, tüm nesneler) her güncellemede istemcilere göndermesidir.

2.1. Nasıl Çalışır?

```
Sunucu (Her Tick'te):
    - Oyun dünyasını güncelle (fizik, input'ları işle)
    - Tüm oyun durumunu (Full State) bir pakete koy
    - Bu paketi bağlı tüm istemcilere gönder

İstemci (Her Tick'te):
    - Sunucudan gelen Full State'i al
    - Mevcut yerel dünyayı tamamen bu yeni durumla değiştir (override)
    - Ekranı render et
```

2.2. Örnek Full State Paketi (JSON)

```
{
  "type": "FULL_STATE_UPDATE",
  "tick": 12345,
  "gameTime": 180.5,
  "players": [
    {
      "id": 1,
      "x": 125.3,
      "y": 75.0,
      "z": 0.0,
      "rotation": 90.0,
      "health": 80,
      "speed": 5.0,
      "weapon": "pistol"
    },
    {
      "id": 2,
      "x": 30.1,
      "y": 120.5,
      "z": 0.0,
      "rotation": 270.0,
      "health": 100,
      "speed": 4.5,
      "weapon": "rifle"
    }
  ],
  "objects": [
    {
      "id": 101,
      "type": "door",
      "state": "open",
      "x": 50.0,
      "y": 50.0
    }
  ],
  "score": {
    "teamA": 5,
    "teamB": 3
  }
}
```

2.3. Full State Sync - Avantajlar ve Dezavantajlar

Avantajlar	Dezavantajlar
Kod basitliği: İstemci, her gelen paketi doğrudan uygular. Eski verilerle karıştırma riski yoktur.	**Bant genişliği tüketimi:** Her pakette tüm veriler gönderilir. Oyuncu sayısı ve nesne sayısı arttıkça paket boyutu büyür.
Hata toleransı: Bir paket kaybolsa bile, bir sonraki tick'te tüm durum tekrar gelir.	**Ölçeklenebilirlik sorunu:** 100 oyunculu bir oyunda full state göndermek neredeyse imkansızdır.

Avantajlar	Dezavantajlar
Geç bağlanan oyuncular: Oyunun ortasında bağlanan bir oyuncu, hemen tam durumu alır ve oyuna dahil olabilir.	**Gereksiz veri:** Her tick'te değişmeyen veriler (örneğin karakter modeli) defalarca gönderilir.
Hata ayıklama (debug): Sunucunun gönderdiği veri ile istemcinin gördüğü veri birebir aynıdır.	**Gecikme (latency):** Büyük paketler daha uzun sürede iletilir, bu da gecikmeyi artırır.

2.4. Hangi Oyunlar Full State Sync Kullanır?

- **Küçük oyunlar:** 2-8 oyuncu arası, basit oyun mekanikleri.
- **Tablı strateji oyunları:** Her birimin durumu kritiktir ve değişim sıktır.
- **Turn-based (sıra tabanlı) oyunlar:** Her turda tüm durum gönderilebilir.
- **Parti oyunları:** Az sayıda oyuncu, basit grafikler.

---

3. Update Frequency (Tick Rate) ve State Sync İlişkisi

Tick rate, state sync'in sıklığını ve dolayısıyla oyunun ne kadar "güncel" göründüğünü belirler.

3.1. Tick Rate ve Bant Genişliği Hesaplama

- **Full State Boyutu:** 100 oyuncu için yaklaşık 2 KB (tahmini).
- **Tick Rate = 20:** Saniyede 20 * 2 KB = **40 KB/s** bant genişliği (sadece state gönderimi).
- **Tick Rate = 60:** Saniyede 60 * 2 KB = **120 KB/s** bant genişliği.

Tick Rate	Bant Genişliği (100 oyuncu)	Duyarlılık (Responsiveness)
10	20 KB/s	Düşük, hareketler geç algılanır
20	40 KB/s	Orta, MMORPG'ler için yeterli
30	60 KB/s	İyi, yavaş FPS'ler için uygun
60	120 KB/s	Çok iyi, rekabetçi oyunlar için ideal
128	256 KB/s	Profesyonel seviye, çok hızlı aksiyon

3.2. Tick Rate Seçimi - Oyun Türüne Göre

Oyun Türü	Önerilen Tick Rate	Neden?
MMORPG (World of Warcraft)	20-30	Hareketler yavaş, büyü (spell) animasyonları uzun. Bant genişliği önemli.
MOBA (LoL, Dota 2)	30-40	Yetenekler anlık ama hedefleme hassasiyeti FPS kadar kritik değil.
Strateji (Age of Empires)	20-30	Birimler toplu hareket eder, bireysel hassasiyet düşük.

Oyun Türü	Önerilen Tick Rate	Neden?
FPS (CS:GO, Valorant)	60-128	Hedef alma ve atış hassasiyeti milisaniyelere bağlı. Yüksek tick rate şart.
Dövüş (Fighting)	60-120	Frame-perfect (kare hassasiyeti) önemli. Her frame senkronize olmalı.

3.3. Tick Rate ve Client-Server Zaman Uyumsuzluğu

İstemci ve sunucu arasında her zaman bir gecikme (latency) vardır. Bu nedenle, istemcinin aldığı state, sunucunun gönderdiği andaki durumdur. Bu durum, istemciye ulaştığında zaten "eski"dir.

- **Örnek:** Sunucu tick'te (T=0) state gönderir. İstemci bu state'i T=50ms'de alır. Bu durumda istemci, 50ms önceki oyun dünyasını görüntülüyordur.
- **Çözüm:** İleri seviye teknikler (Client-Side Prediction, Lag Compensation) bu gecikmeyi tolere eder.

---

4. Uygulama: Server'dan Tüm Oyuncu Pozisyonlarını Broadcast Etme

Bu uygulamada, bir önceki derste oluşturduğumuz sistemi "Full State Sync" mantığına uygun şekilde düzenleyeceğiz. Sunucu, her tick'te tüm oyuncuların konumlarını içeren bir "FULL_STATE_UPDATE" mesajını broadcast edecek.

4.1. Full State Sync Sunucu Kodu (server_full_state.py)

```
import socket
import json
import time

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

class FullStateServer:
    def __init__(self):
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.server_socket.bind((HOST, PORT))
        self.clients = {}  # {addr: {"id": int, "x": float, "y": float,
"last_update": time}}
        self.next_player_id = 1
        self.tick_rate = 30  # Saniyede 30 güncelleme
        self.tick_interval = 1.0 / self.tick_rate
        self.tick_counter = 0
        print(f"Full State Sunucusu {HOST}:{PORT} adresinde çalışıyor...")
        print(f"Tick Rate: {self.tick_rate} Hz")

    def run(self):
        """Ana sunucu döngüsü."""
        last_tick_time = time.time()
```

```
        while True:
            current_time = time.time()

            # Tick işlemleri
            if current_time - last_tick_time >= self.tick_interval:
                self.tick()
                last_tick_time = current_time
                self.tick_counter += 1

            # Non-blocking mesaj alma
            self.server_socket.settimeout(0.005)
            try:
                data, addr = self.server_socket.recvfrom(BUFFER_SIZE)
                self.handle_message(data, addr, current_time)
            except socket.timeout:
                continue
            except Exception as e:
                print(f"Hata: {e}")

    def handle_message(self, data, addr, current_time):
        """Gelen mesajı işler."""
        try:
            message = json.loads(data.decode('utf-8'))
            msg_type = message.get('type')

            if msg_type == 'JOIN':
                # Yeni oyuncu katılımı
                player_id = self.next_player_id
                self.next_player_id += 1
                self.clients[addr] = {
                    "id": player_id,
                    "x": 0.0,
                    "y": 0.0,
                    "last_update": current_time
                }
                print(f"[JOIN] Oyuncu {player_id} katıldı! (Address:
{addr})")

                # Yeni oyuncuya kendi ID'sini gönder
                response = {
                    "type": "ASSIGN_ID",
                    "playerId": player_id,
                    "tickRate": self.tick_rate
                }
                self.server_socket.sendto(json.dumps(response).encode('utf-
8'), addr)

                # FULL STATE gönder (yeni oyuncu hemen güncel durumu alsın)
                self.broadcast_full_state()

            elif msg_type == 'POSITION_UPDATE':
                # Konum güncellemesi
                if addr in self.clients:
```

```
                    self.clients[addr]["x"] = message.get('x', 0.0)
                    self.clients[addr]["y"] = message.get('y', 0.0)
                    self.clients[addr]["last_update"] = current_time

            elif msg_type == 'DISCONNECT':
                # Oyuncu ayrılıyor
                if addr in self.clients:
                    player_id = self.clients[addr]["id"]
                    print(f"[LEAVE] Oyuncu {player_id} ayrıldı.")
                    del self.clients[addr]
                    self.broadcast_full_state()

            elif msg_type == 'GET_STATE':
                # İstemci manuel olarak state istiyor (örneğin yeniden
bağlanma)
                self.broadcast_full_state()

        except json.JSONDecodeError:
            print(f"Geçersiz JSON: {data}")
        except Exception as e:
            print(f"Mesaj işleme hatası: {e}")

    def broadcast_full_state(self):
        """Tüm oyun durumunu FULL olarak tüm istemcilere gönderir."""
        if len(self.clients) == 0:
            return

        # Full state paketini oluştur
        players_list = []
        for addr, player_data in self.clients.items():
            players_list.append({
                "id": player_data["id"],
                "x": player_data["x"],
                "y": player_data["y"]
            })

        full_state = {
            "type": "FULL_STATE_UPDATE",
            "tick": self.tick_counter,
            "timestamp": int(time.time() * 1000),
            "playerCount": len(players_list),
            "players": players_list,
            # Oyun içi diğer veriler buraya eklenebilir
            "gameState": {
                "status": "playing",
                "timeLeft": 300  # Örnek
            }
        }

        state_bytes = json.dumps(full_state).encode('utf-8')
        state_size = len(state_bytes)

        # Tüm istemcilere gönder
        for addr in self.clients.keys():
```

```
            try:
                self.server_socket.sendto(state_bytes, addr)
            except Exception as e:
                print(f"Broadcast hatası ({addr}): {e}")

        # Debug: Her 30 tick'te bir bilgi yazdır
        if self.tick_counter % 30 == 0:
            print(f"[Tick {self.tick_counter}] Broadcast:
{len(self.clients)} oyuncu, {state_size} byte")

    def tick(self):
        """Her tick'te yapılacak işlemler."""
        # Zaman aşımı kontrolü (30 saniye güncellenmemiş oyuncuları
temizle)
        current_time = time.time()
        timeout_seconds = 30.0
        disconnected = []

        for addr, player_data in self.clients.items():
            if current_time - player_data["last_update"] > timeout_seconds:
                disconnected.append(addr)
                print(f"[TIMEOUT] Oyuncu {player_data['id']} zaman aşımına
uğradı.")

        for addr in disconnected:
            del self.clients[addr]

        # Eğer oyuncu ayrıldıysa veya en az 1 oyuncu varsa full state
gönder
        if len(self.clients) > 0:
            self.broadcast_full_state()

if __name__ == "__main__":
    server = FullStateServer()
    try:
        server.run()
    except KeyboardInterrupt:
        print("\nSunucu kapatılıyor...")
        server.server_socket.close()
```

4.2. Full State Sync İstemci Kodu (client_full_state.py)

İstemci, sunucudan gelen full state paketlerini alır ve mevcut oyun dünyasını tamamen bu verilerle günceller.

```
import socket
import json
import threading
import time
import keyboard

HOST = '127.0.0.1'
```

```
PORT = 12345
BUFFER_SIZE = 4096  # Full state daha büyük olabilir, buffer'ı artırdık

class FullStateClient:
    def __init__(self):
        self.client_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.player_id = None
        self.local_x = 0.0
        self.local_y = 0.0
        self.players = {}  # {player_id: {"x": float, "y": float}}
        self.running = True
        self.tick_rate = 30
        self.tick_interval = 1.0 / self.tick_rate
        self.last_state_tick = 0
        self.state_received = False

        # Sunucuya bağlan
        self.connect_to_server()

    def connect_to_server(self):
        """Sunucuya JOIN mesajı gönderir ve ID bekler."""
        join_message = {"type": "JOIN"}
        self.client_socket.sendto(json.dumps(join_message).encode('utf-8'),
(HOST, PORT))
        print("Sunucuya bağlanılıyor...")

        # ID almak için bekle
        self.client_socket.settimeout(3.0)
        try:
            data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
            response = json.loads(data.decode('utf-8'))
            if response.get('type') == 'ASSIGN_ID':
                self.player_id = response['playerId']
                self.tick_rate = response.get('tickRate', 30)
                self.tick_interval = 1.0 / self.tick_rate
                print(f"Oyuncu ID: {self.player_id} atandı. (Tick Rate:
{self.tick_rate})")
            else:
                print("Beklenmeyen yanıt:", response)
                self.running = False
        except socket.timeout:
            print("Sunucudan yanıt alınamadı!")
            self.running = False
        except Exception as e:
            print(f"Bağlantı hatası: {e}")
            self.running = False

    def send_position(self):
        """Mevcut konumu sunucuya gönderir."""
        if self.player_id is None:
            return

        position_message = {
```

```
            "type": "POSITION_UPDATE",
            "x": self.local_x,
            "y": self.local_y
        }

        try:
            self.client_socket.sendto(
                json.dumps(position_message).encode('utf-8'),
                (HOST, PORT)
            )
        except Exception as e:
            print(f"Konum gönderme hatası: {e}")

    def receive_messages(self):
        """Sunucudan gelen FULL_STATE mesajlarını sürekli dinler."""
        self.client_socket.settimeout(0.05)
        while self.running:
            try:
                data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
                message = json.loads(data.decode('utf-8'))
                self.handle_server_message(message)
            except socket.timeout:
                continue
            except json.JSONDecodeError:
                continue
            except Exception as e:
                print(f"Alım hatası: {e}")
                break

    def handle_server_message(self, message):
        """Sunucudan gelen mesajı işler."""
        msg_type = message.get('type')

        if msg_type == 'FULL_STATE_UPDATE':
            # FULL STATE: Tüm oyuncu verilerini tamamen yenile
            self.last_state_tick = message.get('tick', 0)
            players = message.get('players', [])

            # Mevcut player listesini temizle ve yeniden doldur
            self.players.clear()
            for player in players:
                self.players[player['id']] = {
                    'x': player['x'],
                    'y': player['y']
                }

            # Eğer kendi ID'miz listede yoksa (anomaly), kendimizi
ekleyelim
            if self.player_id not in self.players:
                self.players[self.player_id] = {
                    'x': self.local_x,
                    'y': self.local_y
                }
```

```
                self.state_received = True

                # Debug: Sadece her 10 state'te bir yazdır
                if self.last_state_tick % 10 == 0:
                    self.debug_print_state()

            elif msg_type == 'ASSIGN_ID':
                self.player_id = message['playerId']
                print(f"Oyuncu ID güncellendi: {self.player_id}")

        def debug_print_state(self):
            """Mevcut oyun durumunu konsola yazdırır."""
            print("\033c", end="")
            print(f"=== FULL STATE SYNC ===")
            print(f"Oyuncu ID: {self.player_id} | Tick:
    {self.last_state_tick}")
            print(f"Ben: X={self.local_x:.1f}, Y={self.local_y:.1f}")
            print("\n--- Tüm Oyuncular ---")
            for pid, pos in sorted(self.players.items()):
                is_me = " (BEN)" if pid == self.player_id else ""
                print(f"Oyuncu {pid}: X={pos['x']:.1f}, Y={pos['y']:.1f}
    {is_me}")
            print("\n[WASD] Hareket | [Q] Çıkış | [R] State İste")

        def handle_input(self):
            """Klavye girdilerini işler."""
            move_speed = 3.0
            last_send_time = 0
            last_debug_time = 0

            while self.running:
                current_time = time.time()

                # Hareket inputları
                moved = False
                if keyboard.is_pressed('w'):
                    self.local_y += move_speed * self.tick_interval
                    moved = True
                if keyboard.is_pressed('s'):
                    self.local_y -= move_speed * self.tick_interval
                    moved = True
                if keyboard.is_pressed('a'):
                    self.local_x -= move_speed * self.tick_interval
                    moved = True
                if keyboard.is_pressed('d'):
                    self.local_x += move_speed * self.tick_interval
                    moved = True

                # R tuşu ile manuel state isteği
                if keyboard.is_pressed('r'):
                    print("State isteği gönderiliyor...")
                    self.client_socket.sendto(
                        json.dumps({"type": "GET_STATE"}).encode('utf-8'),
                        (HOST, PORT)
```

```
                )
                time.sleep(0.3)  # Çok fazla istek göndermeyi önle

            # Q ile çıkış
            if keyboard.is_pressed('q'):
                print("Çıkılıyor...")
                self.disconnect()
                break

            # Konumu tick rate'de gönder
            if moved or (current_time - last_send_time >=
self.tick_interval):
                self.send_position()
                last_send_time = current_time

            # Her saniye debug yazdır (eğer state gelmediyse)
            if not self.state_received and (current_time - last_debug_time
>= 1.0):
                print("Sunucudan state bekleniyor...")
                last_debug_time = current_time

            time.sleep(0.01)

    def disconnect(self):
        """Sunucudan ayrılma mesajı gönderir ve bağlantıyı kapatır."""
        self.running = False
        if self.player_id is not None:
            disconnect_message = {"type": "DISCONNECT"}
            try:
                self.client_socket.sendto(
                    json.dumps(disconnect_message).encode('utf-8'),
                    (HOST, PORT)
                )
            except:
                pass
        self.client_socket.close()
        print("Bağlantı kapatıldı.")

    def run(self):
        """İstemciyi çalıştırır."""
        if not self.running:
            return

        print("İstemci başlatıldı. WASD ile hareket edin.")
        print("R tuşu ile manuel state isteği gönderebilirsiniz.")
        print("Q tuşuna basarak çıkabilirsiniz.")

        # Mesaj alma thread'i
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

        # Input işleme (ana thread)
        try:
```

```
            self.handle_input()
        except KeyboardInterrupt:
            print("\nKullanıcı tarafından durduruldu.")
        finally:
            self.disconnect()

if __name__ == "__main__":
    client = FullStateClient()
    client.run()
```

---

5. Full State Sync'in Önemli Detayları

5.1. Packet Boyutu Optimizasyonu

Yukarıdaki örnekte JSON kullanıyoruz. 10 oyuncu için paket boyutu yaklaşık 500-800 byte. Ancak 100 oyuncu için bu 5-8 KB'a çıkar. Optimizasyon için:

1. **Binary format kullanın:** JSON yerine binary ile boyutu %70 azaltabilirsiniz.
2. **Sadece değişen verileri gönderin:** Full state yerine delta (fark) gönderimi yapın.
3. **Sıkıştırma:** Zlib veya benzeri sıkıştırma kullanın.

5.2. State Version (Sürüm) Yönetimi

Her full state paketine bir `tick` veya `version` numarası ekleyin. İstemci, eski bir state'i (örneğin gecikmeli gelen paket) alırsa, onu görmezden gelebilir.

```
# İstemci tarafı
if message.get('tick', 0) <= self.last_state_tick:
    # Bu eski bir state, atla
    return
self.last_state_tick = message.get('tick')
```

5.3. Partial Update (Kısmi Güncelleme)

Full state gönderirken bile, sadece değişen oyuncuların verilerini göndermek mantıklıdır. Ancak bu, "Full State" tanımına tam olarak uymaz (Delta State'e girer).

5.4. Geç Bağlanan Oyuncular (Late Joiners)

Full state sync'in en büyük avantajı, oyunun ortasında bağlanan bir oyuncunun hemen tüm durumu alabilmesidir. Sunucu, yeni oyuncuya JOIN yanıtı olarak hemen bir full state gönderir.

---

6. Delta State Sync ile Karşılaştırma (Ön Bilgi)

Özellik	Full State Sync	Delta State Sync
Gönderilen Veri	Tüm oyun dünyası	Sadece değişen veriler
Paket Boyutu	Büyük	Küçük

Özellik	Full State Sync	Delta State Sync
Bant Genişliği	Yüksek	Düşük
Hata Toleransı	Yüksek (bir sonraki paket her şeyi düzeltir)	Düşük (paket kaybı veri tutarsızlığına yol açar)
Kod Karmaşıklığı	Düşük	Yüksek
Kullanım Alanı	Küçük oyunlar, prototip, lobi	Büyük oyunlar, MMO, FPS

Delta State Sync, ileri seviye bir konudur ve bir sonraki derste detaylı olarak işlenecektir.

İleri Çalışma

1. **Oyuncu Sayısını Artırın:** Aynı anda 5-10 istemci çalıştırarak sistemin nasıl davrandığını gözlemleyin.
2. **Paket Boyutu Analizi:** Gönderilen her `FULL_STATE_UPDATE` paketinin byte boyutunu sunucuda loglayın. Oyuncu sayısı arttıkça boyutun nasıl değiştiğini grafikleştirin.
3. **Binary Dönüşümü:** Full state paketini JSON yerine binary (struct.pack) ile gönderecek şekilde dönüştürün. Paket boyutundaki farkı ölçün.
4. **State Geçerlilik Kontrolü:** İstemciye `tick` numarasını gösterin. Eğer gelen state'in tick'i bir öncekinden düşükse (eskiyse) bu state'i uygulamasın.
5. **Color Coding:** Konsol çıktısında kendi oyuncunuzu farklı renk (veya işaret) ile gösterin.

Özet

- **State Sync:** Oyun dünyasının sunucudan istemcilere iletilmesi sürecidir.
- **Full State Sync:** Tüm oyun durumunun her tick'te gönderilmesidir. Basit, hata toleransı yüksek ama bant genişliği tüketir.
- **Tick Rate:** Güncelleme sıklığı. Yüksek tick rate daha duyarlı ama daha fazla bant genişliği gerektirir. Oyun türüne göre optimizasyon yapılmalıdır.
- **Broadcast:** Sunucunun aynı veriyi tüm istemcilere göndermesidir.
- **JSON vs Binary:** Prototipleme için JSON, verimlilik için binary tercih edilmelidir.

Full State Sync, multiplayer oyun geliştirmeye başlarken en kolay uygulanabilen yöntemdir. Oyununuz büyüdükçe ve oyuncu sayısı arttıkça, Delta State Sync ve diğer optimizasyon tekniklerine geçiş yapmanız gerekecektir.